Blockchain: A Ground-Up Mastery Guide

Shaheer Ahmad

June 21, 2026

Contents

Phase 1: Foundations

The Core Problem Blockchain Solves

Every financial system in history has relied on a **trusted middleman** — a bank, a government, a court — to maintain a record of who owns what. When you send money, you are not moving anything physical; you are asking your bank to update a number in *their private database*.

The fundamental vulnerabilities of this model:

- The middleman can be corrupt or coerced.
- The middleman is a single point of failure.
- Access can be denied or frozen.
- Cross-border trust is fragile.

Satoshi Nakamoto's insight (2008): Replace the single trusted middleman with *thousands of strangers* each holding an identical copy of the same ledger, all of whom must agree before any change is accepted. This is the blockchain.

Traditional System	Blockchain
One private database	Thousands of identical copies
Trust the institution	Trust the math
Single point of failure	No central point to attack
Permission required	Open and permissionless

Why "Chain"? The Structure of a Blockchain

A blockchain is composed of **blocks** linked in a chain. Each block contains:

1. A set of **transactions** (the data).
2. The **hash of the previous block** — this is the "chain" link.
3. Its own **hash** (a fingerprint of its entire contents).

```
BLOCK #2                    BLOCK #3
Txns: A→B: 500              Txns: B→C: 200
Prev Hash: [Block1]   Prev Hash: [Block2]
Own Hash:  a3f2c7..         Own Hash:  7b84fa..
```

Why chaining prevents tampering: If an attacker modifies any transaction in Block 2, Block 2's hash changes entirely. This invalidates Block 3's "Prev Hash" reference, which cascades and breaks every subsequent block. The fraud becomes immediately visible to all participants.

Two distinct ideas work together:

Mechanism	What It Prevents
Chaining (linked hashes)	Secretly editing old history
Distribution (thousands of copies)	Any single party controlling the record

The Hash Function: The Mathematical Fingerprint

A **hash function** takes any input and produces a fixed-size output (the hash). Bitcoin uses **SHA-256**, which always produces a 256-bit output.

The four critical properties:

1. **Deterministic:** The same input always produces the same hash. $H(\text{“input”}) \rightarrow$ always the same output.
2. **Avalanche Effect:** A single character change causes the entire hash to change completely and unpredictably. There is no gradual drift.
3. **Fixed Size:** Regardless of whether the input is one word or one million pages, the output is always the same length (256 bits for SHA-256).
4. **One-Way (Pre-image Resistance):** Given a hash h, it is computationally infeasible to find an input x such that $H(x) = h$. You cannot reverse-engineer the original data from the fingerprint.

Example — SHA-256 Avalanche Effect:

Input	`SHA-256 Hash (truncated)`
“Ali owes Shaheer 1000”	`a3f2c7d1...9e8b`
“Ali owes Shaheer 1001”	`7b84fa3c...2d07`

One digit changed. The hash is completely unrecognisable.

Why the one-way property is security-critical:

- **Privacy:** No one can read transaction details by inspecting a hash.
- **Forgery prevention:** An attacker who wants to secretly alter Block 2 while keeping its hash identical (so Block 3 remains valid) would need to *reverse* the hash function to engineer a collision. The one-way property makes this computationally infeasible, rendering such forgery impossible.

Formally, a secure hash function satisfies **collision resistance**: it is infeasible to find two distinct inputs $x \neq x'$ such that $H(x) = H(x')$.

Phase 2: Consensus Mechanisms

The Core Problem: Who Gets to Add Blocks?

With thousands of strangers on the network, none of whom trust each other, a critical question arises: who has the authority to append the next block? Appointing a gatekeeper would recreate the centralisation problem. Allowing anyone to add freely would enable spam and fraud.

The solution requires a mechanism with one key property:

Hard to produce. Trivially easy to verify.

- **Hard to produce** $\Rightarrow$ Adding a block costs real effort. A cheater cannot flood the network with fake blocks cheaply.

- **Easy to verify** ⇒ Every node can confirm validity in milliseconds without trusting the producer.

Sybil Attack: If block creation were cheap, a single attacker could masquerade as thousands of nodes and dominate the network. The cost of production is the defence.

Proof of Work (PoW)

Bitcoin's consensus mechanism. The "puzzle" is defined using the hash function:

> Find a number n (called a **nonce**) such that:
>
> $$H(\text{block data} \parallel n) < T$$
>
> where T is a target threshold, equivalent to requiring the hash output to begin with a certain number of leading zeros.

Example:

```
H(block_data + nonce) = 00000000003a7f2c...    valid
H(block_data + nonce) = 7b84fa3c9d2e1a05...    invalid
```

Why this works:

- Because hashes are **one-way**, there is no shortcut. You cannot calculate the correct nonce — you must guess, billions of times, until a valid hash appears by chance.
- Because hashes are **deterministic**, anyone can verify the answer by running the hash function exactly once.

The process of competing to find this nonce is called **mining**.

The Incentive Structure: Why Miners Are Honest

A miner who wins (finds the valid nonce) earns:

1. **Block reward** — newly minted Bitcoin, created from nothing. This is the only mechanism by which new Bitcoin enters circulation.
2. **Transaction fees** — a small fee attached to every transaction by its sender.

This creates a rational incentive table:

Miner Action	Outcome
Valid block + transactions included	Block reward + all fees ✓
Valid block + empty block	Block reward only (fees forfeited)
Valid block + fraudulent transactions	Block rejected, zero reward, electricity wasted ×

Key insight (Mechanism Design): Satoshi did not ask miners to be honest. He constructed the reward structure so that honesty is the *most profitable rational strategy.* The system enforces good behaviour through economics, not trust.

Difficulty Adjustment

The target threshold T is adjusted automatically every 2016 blocks ($\approx$ 2 weeks) so that the average time between blocks stays at **10 minutes**, regardless of how much total computing power joins or leaves the network.

More miners $\Rightarrow$ puzzles get harder. Fewer miners $\Rightarrow$ puzzles get easier.

The 51% Attack

If a single entity controls more than half the network's total computing power (hashrate), they can:

- Consistently solve the puzzle faster than honest miners.
- Propose a secret alternative chain and eventually make it longer.
- Rewrite recent history (double-spend attacks).

This is called a **51% attack**. Bitcoin's security therefore scales directly with the total honest hashrate on the network. The larger and more distributed the network, the more astronomically expensive such an attack becomes.

Phase 3: Proof of Stake & Alternative Consensus

The Problem with Proof of Work

PoW is deliberately wasteful. Billions of hash guesses are computed solely to produce a lottery ticket — the actual computation creates no useful output. Bitcoin's annual energy consumption rivals that of entire countries.

Research question: Can we replace "burning electricity" with something else as proof of legitimate participation?

Proof of Stake (PoS)

Instead of expending energy, validators **lock up** (stake) their coins as collateral. The protocol pseudo-randomly selects a validator to propose the next block, weighted by stake size:

$$P(\text{selected}) \propto \text{stake}_i$$

Punishment — Slashing: If a validator is caught proposing fraudulent blocks or signing contradictory blocks, the protocol automatically **destroys** a portion of their staked coins. The attacker does not merely earn nothing — they lose what they committed.

Property	**Proof of Work**	**Proof of Stake**
Right to add block	Burn electricity	Lock up coins
Punishment for cheating	Wasted electricity	Slashing (coins destroyed)
Energy use	Enormous	$\sim$99.9% less
Hardware required	Specialised ASICs	Ordinary computer
Attack cost	Buy 51% of hashrate	Buy 51% of all staked coins

Wealth concentration problem: A validator's selection probability scales with stake. This means wealthier participants earn more rewards, compounding their advantage — a genuine ongoing tension in PoS system design.

Ethereum migrated from PoW to PoS in September 2022 ("The Merge").

Delegated Proof of Stake (dPoS)

Motivation: In plain PoS, hundreds of validators must coordinate to reach consensus — this is slow. dPoS introduces representative democracy to the blockchain.

Mechanism:

1. Token holders **vote** to elect a small fixed set of trusted **delegates** (e.g. 21 in EOS).
2. Only elected delegates produce blocks, taking turns in a round-robin schedule.
3. Token holders can vote out a misbehaving delegate at any time.

Parliament analogy: Token holders are citizens (voters); delegates are elected MPs. Only MPs pass laws (add blocks), but citizens can replace them at any election.

Property	dPoS Advantage
Speed	Small delegate set ⇒ fast consensus
Accountability	Delegates voted out instantly for misbehaviour
Participation	Anyone can vote regardless of stake size

Tradeoff: Trust is concentrated in a small delegate group — more centralised than pure PoS or PoW.

Two mechanisms that constrain a malicious delegate:

1. **Social (Voting):** Token holders immediately vote the delegate out, terminating all future block rewards and their position permanently.
2. **Economic (Slashing):** Provable misbehaviour destroys the stake the delegate deposited to obtain their position. They paid to cheat and lose both the stake and the role.

CeDAR relevance: The lab's paper on blockchain-enabled data sharing in Connected Autonomous Vehicle (CAV) networks used dPoS with reputation schemes (Multi-Weight Subjective Logic, beta, sigmoid) layered on top to detect malicious nodes — addressing the centralisation tradeoff through reputation scoring before a node is even eligible for delegation.

Phase 4: Smart Contracts & Ethereum

The Core Idea

Bitcoin's blockchain records *who paid whom*. Ethereum extended this: what if the blockchain could also *run code*?

A **smart contract** is a program deployed permanently on the blockchain. No one owns it, no one can stop it, and no one can modify it after deployment. When its conditions are met, it executes automatically and irreversibly.

Two fundamental limitations (discovered through reasoning):

1. **Incomplete contracts:** Code only handles what the programmer anticipated. Real-world edge cases (a draw, a disputed result, a force majeure) have no `else` clause. A judge can use judgment to fill legal gaps; a smart contract cannot. This is the **"Code is Law" problem.** Famous example: *The DAO Hack (2016)* — an attacker drained $60M by exploiting the exact code as written. No rules were broken, because the code *was* the rules.

2. **The Oracle Problem:** A blockchain is a sealed, self-contained system. It cannot read the internet, watch a match, or sense the real world. Any contract that depends on external data requires a trusted data feed called an **oracle**. But trusting a single oracle re-introduces a centralised middleman. Solutions like **Chainlink** use decentralised oracle networks with their own staking/slashing mechanisms to make lying economically irrational.

Storage and Execution: The EVM

Deployment: A smart contract is deployed by sending a special transaction whose payload contains the compiled contract bytecode. The network stores this code permanently at a unique **contract address**. It cannot be deleted.

Execution — The Ethereum Virtual Machine (EVM): Every node runs an identical virtual machine. When a contract is called, every node executes the same bytecode independently and must arrive at the same result. Disagreement results in rejection. This mandates that smart contracts be **deterministic** — given the same inputs, they must always produce the same output. Non-deterministic operations (random numbers, system time, external calls) are forbidden without special handling.

Gas: Metered Computation

The halting problem: A contract with an infinite loop would cause every node to compute forever, halting the network.

Solution — Gas: Every EVM instruction costs a fixed number of gas units. The caller pays upfront. Execution stops when gas is exhausted.

$$\text{Transaction Cost (ETH)} = \text{Gas Used} \times \text{Gas Price}$$

Gas serves three simultaneous functions:

1. **Prevents infinite loops** — execution halts when funds run out.

2. **Prioritises transactions** — validators include high-gas-price transactions first (rational economic choice).

3. **Compensates validators** — gas fees are the transaction fees validators collect per block.

Access Control: The Foundational Security Pattern

The single most important security pattern in smart contract development:

```
require(msg.sender == authorisedAddress, "Unauthorised");
```

What it does: Before executing any logic, verify *who* is calling. `msg.sender` is the cryptographically verified address of the caller — it cannot be forged. If the check fails, the entire transaction reverts as if it never occurred.

What it prevents: Any attacker who tries to impersonate an authorised party (e.g. an oracle, an owner, a regulator) to trigger fraudulent contract execution — such as calling `settle(true)` on a bet contract to falsely claim a payout.

Example — Cricket Bet Contract:

```
contract CricketBet {
    address payable public shaheer;
    address payable public ali;
    address public oracle;

    constructor(address payable _ali, address _oracle) payable {
        shaheer = payable(msg.sender); // deployer
        ali     = _ali;
        oracle  = _oracle;
    }

    function settle(bool pakistanWon) external {
        // Only the trusted oracle may call this
        require(msg.sender == oracle, "Only oracle can settle");
        if (pakistanWon) {
            shaheer.transfer(address(this).balance);
        } else {
            ali.transfer(address(this).balance);
        }
        // Note: draw case unhandled -- incomplete contract problem
    }
}
```

Key Solidity primitives:

- `msg.sender` — cryptographically verified caller address.
- `require()` — guard clause; reverts entire transaction on failure.
- `transfer()` — sends ETH atomically; no bank required.
- `address(this).balance` — the ETH held by the contract itself.

CeDAR Relevance: Smart Contracts as Policy Enforcers

CeDAR's core research encodes complex organisational access-control policies (written in XACML) as smart contracts on Ethereum. The `require()` pattern scales from a single address check to full **attribute-based access control (ABAC)**: before granting a doctor access to a patient's record, the contract verifies role, department, time-of-day, patient consent, and audit trail — all on-chain, all without a trusted administrator.

The gas cost of evaluating these policies on-chain is a primary research challenge. CeDAR's work on *policy composition* (merging multiple organisations' policies into one composite smart contract) is specifically motivated by minimising on-chain gas expenditure.

Phase 5: Cryptographic Identity — Digital Signatures

The Core Problem: Proving Authorship Without a Middleman

When you send a transaction on Bitcoin, you broadcast a message to thousands of strangers. How does every node on earth know that *you* — and not an impersonator — authorised it? There is no username, no password, no bank to call. The answer is **public-key cryptography**.

The Magic Padlock: Asymmetric Keys

A normal padlock uses one key to both lock and unlock (symmetric). The problem: to give someone a locked box, you must first share the key — but how do you share it safely?

The solution is a padlock with a fundamentally different property:

> It comes with **two different keys**: a Private Key and a Public Key.
> What the Private Key *locks*, only the Public Key can *unlock*.
> Knowing the Public Key reveals *nothing* about the Private Key.

You keep the **Private Key** (sk) secret forever. You give the **Public Key** (pk) to everyone — post it publicly.

How signing works:

1. You lock your message with your *private* key and send it.
2. Anyone uses your *public* key to open it.
3. The fact that the public key opened it *proves* the private key locked it — and only you have the private key.

The unforgeability guarantee: Even if an attacker knows your public key (everyone does), they cannot work backwards to find your private key. The relationship between the two keys is one-way.

Key	Kept by	Operation
Private key sk	Owner only (secret)	**Sign** (lock) a message
Public key pk	Everyone (public)	**Verify** (unlock) a message

The Mathematical Foundation: Modular Arithmetic

All of this cryptography is built on **modular (clock) arithmetic**.

A 12-hour clock wraps around at 12:

$$15 \equiv 3 \pmod{12}$$

The Discrete Logarithm Problem: Consider a clock with 7 positions. Start at 1 and repeatedly add 3:

$$1 \to 4 \to 0 \to 3 \to 6 \to 2 \to 5 \to 1 \to \cdots$$

The sequence jumps unpredictably. The question "I added 3 exactly k times and landed on 5 — what is k?" has no algebraic shortcut. You must try every possibility. For real cryptography the clock has $\sim 2^{256}$ positions: finding k requires 2^{128} operations. Computationally impossible.

Elliptic Curve Cryptography (ECC)

Bitcoin and Ethereum use the curve **secp256k1**:

$$y^2 \equiv x^3 + 7 \pmod{p}$$

Points on this curve form a group under **point addition** (draw a line through two points, reflect the third intersection). Key generation:

$$P = k \cdot G = \underbrace{G + G + \cdots + G}_{k \text{ times}}$$

- G — public generator point (known to everyone)
- k — **private key** (random 256-bit integer, secret)
- P — **public key** (point on curve, shared openly)

Forward ($k \to P$): fast, $O(\log k)$ steps via repeated doubling.
Reverse ($P \to k$): the **ECDLP** — requires $O(2^{128})$ operations.

Digital Signatures: ECDSA — The Two Attacks and Their Defeat

Two attacks a forger could attempt:

1. **Forge without private key:** create a valid signature for a fraudulent transaction from scratch.
2. **Replay attack:** copy a valid signature from one transaction and attach it to a different, fraudulent one.

Why hash before signing? The message can be any length; the signing math requires a fixed-size input. Hashing converts any message to exactly 256 bits: one hash, one signature, regardless of message size.

Signing (private key k, message hash m):

1. $m = H(\text{transaction data})$
2. Pick fresh random nonce r; compute $R = r \cdot G$
3. $s = r^{-1}(m + k \cdot R_x) \pmod{n}$
4. Output signature $\sigma = (R_x,\ s)$

Verification (public key $P = kG$, no private key needed):

$$R' = s^{-1} \cdot m \cdot G + s^{-1} \cdot R_x \cdot P$$

Valid iff $R'_x = R_x$.

Why the algebra works (substituting $P = kG$, $s = r^{-1}(m + kR_x)$):

$$R' = \frac{r}{m + kR_x} \cdot m \cdot G + \frac{r}{m + kR_x} \cdot R_x \cdot kG = \frac{r(m + kR_x)}{m + kR_x} \cdot G = r \cdot G = R \checkmark$$

The private key k cancels completely — proven without ever being revealed.

Attack 1 defeated: Forging s requires k. Without it, the verification equation produces $R'_x \neq R_x$.

Attack 2 (replay) defeated: m is baked inside s. Attaching σ to a different message changes m, which changes R'_x. The signature is *message-bound* — physically inseparable from the exact transaction it was created for.

The Nonce Reuse Catastrophe

Critical rule: The nonce r must be freshly random for every signature.

What happens if two signatures share the same r:

Given $s_1 = r^{-1}(m_1 + kR_x)$ and $s_2 = r^{-1}(m_2 + kR_x)$:

$$s_1 - s_2 = r^{-1}(m_1 - m_2) \implies r = \frac{m_1 - m_2}{s_1 - s_2}$$

And once r is known:

$$k = \frac{s_1 \cdot r - m_1}{R_x}$$

The entire private key is exposed. Every coin in the wallet is lost. All future transactions can be forged. The damage is total and permanent.

Real-world disaster — Sony PlayStation 3 (2010): Sony signed all PS3 firmware with the *same* nonce r. Researchers applied the above algebra, recovered Sony's master private key, and could sign arbitrary software as legitimate Sony firmware — bypassing all security on every PS3 ever sold.

Modern fix — RFC 6979: Rather than trusting a random number generator, r is derived *deterministically* from the private key and message hash:

$$r = \text{HMAC-SHA256}(k,\ m)$$

This guarantees r is unique per (key, message) pair without any randomness that could fail or repeat.

Complete Signature Reference Table

Component	Value	Role
Private key k	256-bit secret	Creates the signature
Public key $P = kG$	Point on curve	Verifies the signature
Message hash m	H(transaction)	Binds signature to exact message
Nonce $R = rG$	Fresh random point	Prevents replay; must be unique
Signature (R_x, s)	Two integers	Proof of knowledge of k, bound to m

The Complete Bitcoin Transaction Flow

```
YOUR WALLET
  Private key k  (secret, never leaves device)
         k · G   (one-way: ECDLP)

  Public key P   (shared openly)
         Hash(P)  (one-way: SHA-256 + RIPEMD-160)

  Address         (e.g. 1A2b3C... | what others send funds to)

SENDING A TRANSACTION
  1. Compose:  "Send 500 to Ali"
  2. Hash:      m = H("Send 500 to Ali")
  3. Sign:       = (Rx, s)  using private key k
```

```
  4. Broadcast: (message, , P) → all nodes

EVERY NODE VERIFIES INDEPENDENTLY
  Verify(P, m, ) → true   (no private key needed)
```

Security depth: Two one-way barriers stand between your address and your private key: ECDLP ($k \to P$) then hashing ($P \to$ address). An attacker who sees only your address cannot recover P; even with P they cannot recover k.

Phase 6: DeFi, Tokens & Tokenomics

Coming next: What are tokens? How does DeFi replace banks with mathematics? Stablecoins, AMMs, and the RC-coin from CeDAR's own research.